Viva answers

One-line answers, the design choices and the weak spots

CNN-LSTM Based Spatiotemporal Flood Evolution Forecasting · Anhad Mahajan and Aman Sumeet Saxena · Deep Learning CA3

1 Read this first: how to answer, and the weak spots mam could catch

For any code question: read the comment above the line aloud, say what goes in and what comes out, and name the tool and what it is for. If a line is unfamiliar, say what it is for in the story, and do not guess.

1.1 Things mam could catch, and what to say

These are the weak spots we know about. It is better to say them yourself before she asks.

1. **Slide 10, the linear-model bars (already corrected).** The linear model now shows **0.818, 0.798 and 0.790**, the numbers in the final report. Our first fit gave 0.32, 0.35 and 0.15 because of a scaling mistake (the main guide). If she asks why the number changed, say: "Our first fit had a bug. After the fix a plain linear formula scores 0.82, 0.80 and 0.79, as good as the lookup table or a little better, and the networks add only 0.01 to 0.05. The conclusion does not change."
2. **The model notebook loads saved models, it does not train.** The switch `TRAIN_NEW = False` loads the weights we trained earlier. The `fit` cells hold the training code, and True trains again.
3. **Part 2 (the 310-run laptop study) is not in the Colab notebooks.** It ran on a laptop CPU, took about 20 hours, and we are not showing its code. Slides 10 and 11 show its results, and the code to show is in the three notebooks.
4. **The alignment table says "Both tests say move it" for Australia 30 m and UK 30 m.** The best match is only one 30 m square from our position, the smallest possible move. The report says the two checks agree within about one cell, so we kept the positions.
5. **In the capsule Grad-CAM figure the four columns of each row look alike.** The four tiles are the four most flooded test tiles, most likely the same place at nearby times. The report also says the capsule heat maps were weak and are pictures, not proof.
6. **Red warning text in the outputs** (`NotGeoreferencedWarning`, the Keras `adam` warning, `HTTP Error 429`) is harmless: missing map-position data (saved in `map_info.json`), a saved optimiser memory, and Hugging Face asking us to slow down.
7. **The last cell of the model notebook lists old file names** such as `P1_skill_comparison.png`. They are older runs still in the Drive folder. This run's figures are `03_...` to `06_....`
8. **Results are single runs.** We did not repeat Part 1 with many seeds, and the limits slide says so.

2 One-line answers to the questions mam may ask

Say the bold line. The small print is only for when she asks for more. Questions about each layer and number of the model (why 16 filters, why a second convolution) are in Chapter 3.

2.1 The project and the data

Q. What is your project about?
A. We forecast flood depth maps 30 minutes ahead with a CNN-LSTM and test whether it beats very simple forecasts like copying the last map. Input: the last four maps plus rain that has already fallen plus ground height. Tested on floods the model never saw.

Q. Why this dataset?
A. FloodCastBench gives the water depth every 5 minutes for four floods in four countries at several map sizes, with rain and ground height, so we always know the right answer. Its own benchmark has U-Net and FNO but no recurrent model and no simple-forecast comparison.

Q. Is the data real?
A. It is simulated from real terrain, land cover and satellite rain, so our models learn to imitate the simulator. We say this on the limits slide.

Q. What is the novelty?
A. A careful test of a convolutional-recurrent model on FloodCastBench against simple forecasts on never-seen floods, plus one pipeline with written rules and heat maps. We did not invent a new architecture.

Q. What is your result in one line?
A. A trend forecast with one fitted number scores 0.91 to 0.99 at 30 minutes, matching our network, so always compare a model with simple forecasts. Model A scored 0.15 to 0.35 and -1.449 in late Australia. Model B matches the trend.

Q. Is it useful in real life?
A. Not yet as a warning system, but it is a fair test of whether such a network is worth using, and it answers in seconds while the simulator takes days.

Q. Why CNN-LSTM?
A. The CNN sees neighbouring squares and the LSTM remembers how the water has been moving, and it was the model in our project title.

Q. Why only 30 minutes ahead?
A. Rain maps come every 30 minutes, so it is the natural step and the simplest fair test, and longer forecasts are our next step. The laptop study went up to 2.5 hours.

Q. Why train on Australia and Pakistan, test on UK and Mozambique?
A. The test is a flood in another country that the model never saw, and the 30 m and 480 m maps need separate models.

Q. Why a 70, 15, 15 split?
A. The first 70% of a flood trains, the next 15% decides when to stop, the last 15% tests, and splitting in time makes the test part truly later.

Q. Is there data leakage?

A. No, beta is fitted only on the training part, early stopping uses only the checking part, and the tests use unseen time and unseen floods. In the notebook we did not choose a setting by looking at test scores.

2.2 Score, trend forecast and models

Q. What is skill?
A. One minus the error of the forecast divided by the error of copying the last map, so 0 is the same as copying, 1 is perfect, below 0 is worse. Accuracy does not suit depth maps, which hold a number in every square.

Q. What is CSI?
A. Of the squares flooded (above 30 cm) in the forecast or in reality, the share where both agree.

Q. What is the trend forecast?
A. The last map plus beta times the last change, never below zero, with no network, and beta is 0.921 for 30 m and 0.948 for 480 m.

Q. Model A against Model B?
A. Same layers, 62,833 parameters, but A starts from the last map and B starts from the trend and learns only a small correction.

Q. Why did you build Model B?
A. The trend forecast beat Model A by a lot, so we asked whether a network that starts from the trend could improve on it.

Q. Why does Model B only match the trend?
A. At 30 minutes only 1, 5, 9 and 6% of copying's error is left after the trend, so there is almost nothing to add. A fair test of the memory needs 6 to 24 hours.

Q. Why is Model A negative in Australia?
A. In the last part of that flood the water hardly moves and copying is off by about a millimetre, while Model A adds a little water almost everywhere. We first thought it never saw draining water, but 73 of 331 training steps had falling water.

Q. Skill numbers?
A. Trend 0.990, 0.947, 0.908, 0.942, Model A -1.449, 0.305, 0.345, 0.150, Model B 0.989, 0.948, 0.908, 0.943. Order: Australia last 15%, UK, Pakistan last 15%, Mozambique.

Q. What did the 310-run laptop study show?
A. Up to 2.5 hours the depth history carries almost all the skill: lookup table 0.74 to 0.79, best networks 0.81 to 0.87.

Q. Does the LSTM memory help?
A. Not at equal training length: the difference is $+0.004$ on the UK and -0.014 on Mozambique, inside our ± 0.03 band. With the 30-pass training it lost less skill at longer lead times and stayed stable when the flood start was hidden. We did not test why.

Q. What did the ridge mistake do?
A. It made the linear baseline score 0.15 to 0.36 instead of 0.79 to 0.82, and Aman found and fixed it. Slide 10 shows the corrected numbers.

2.3 NeuroSymbolic AI, capsule network, Grad-CAM

Q. What is NeuroSymbolic AI and what did you do?
A. A neural network checked by written rules people can read: the water-balance rule plus three severity rules, each with a written reason. Rule 1: under 1% flooded now and forecast means None. Rule 2: the class cannot fall below the current one if the forecast is not less flooded and over 1 mm per hour of rain fell in the last hour. Rule 3: if network and forecast map differ by more than one class, the forecast map wins.

Q. What is the water-balance rule?
A. It shifts the forecast so the total rise of the water matches the rain of the last 30 minutes times 0.5 hours. It halved the Australian failure (-1.449 to -0.710) but adds nothing to Model B. The law holds closely, ratios 0.999, 0.946, 1.028, 1.045.

Q. Where is Grad-CAM?
A. In the flood model notebook, Section 6, the function `grad_cam`, shown for the forecaster and for the capsule network.

Q. What is Grad-CAM?
A. A heat map of which places mattered most for an answer, from the gradient of one number with respect to the ConvLSTM memory maps. Bright follows rivers and flood edges. The capsule heat maps were weak, so they are pictures, not proof.

Q. What is a capsule network?
A. A network whose units give a short vector whose length tells how sure it is, with votes combined by routing by agreement.

Q. Did the capsule network work?
A. Partly: 60.9, 50.0, 87.4, 63.5% and with the rules 67.9, 56.1, 89.1, 70.7%, but the forecast map's own class is right about 99%. It did not beat a plain layer once we used augmentation.

Q. Is Pakistan's 87.4% great?
A. No, 698 of the 1,000 test tiles are severe, so always answering severe is right 69.8% of the time.

2.4 Why not, and what next

Q. Why not real flood data?
A. Real depth maps every few minutes over a region do not exist, since satellites pass every few days, so a simulator gives the complete maps.

Q. Why not 6 to 24 hours?
A. It needs forecasts fed back as input and an assumption about future rain, so it is our main next step.

Q. Why not more floods or seeds?
A. FloodCastBench has only four floods, so only two independent train and test pairs, and the Colab results are single runs.

Q. Why not tune more?
A. Differences below about 0.02 were inside seed noise in the laptop study, so tuning could not change the conclusion.

Q. Next steps?
A. Forecast 6 to 24 hours ahead, rebuild the capsule network and Grad-CAM on Model B, and repeat with several seeds.

2.5 The work and the notebooks

Q. What problems did you face?
A. A one-hour 27 GB download with rate limits, aligning the layers, our first model failing against the trend, big maps in Grad-CAM, a bad severity label and the ridge mistake.

Q. Why three notebooks?
A. Downloading and exploring, preparing, and modelling are three big slow jobs, and the prepared data is saved so each can start from files.

Q. Did the notebook train the models?
A. No, it loads the saved models from Drive, and setting `TRAIN_NEW` to True trains them again with the `fit` cell. The notebook does not show the training time. Training stops early, after at most 20 epochs.

Q. Who did what?
A. Aman built and ran the Colab notebooks, Anhad ran the laptop study, both wrote the report, and Aman found the ridge mistake.

Q. How do you know the data preparation is right?
A. Every file arrived, every layer covers the depth map, two independent alignment tests agree, and all 30 saved files are identical to the copy on Drive.

Q. Why is your model worse than a trivial baseline, and why present it?
A. That is the finding: beating copying looked like success until we compared with the trend, and showing it is more useful than hiding it.

Q. Where is the code for the 310 runs?
A. It was a separate 20-hour run on a laptop, its results are on slides 10 and 11, and the code we show you is in the three Colab notebooks.

Q. Why trust a simulator?
A. We do not take it as truth, it follows basic water-flow rules with real ground and rain and is published, and our models only imitate it.

2.6 Reading the code

Q. What does None mean in (None, 4, 64, 64, 3)?
A. It is the batch size, which can be any number, and the rest is one example: 4 maps of 64 by 64 with 3 layers.

Q. What does inputs[:, -1, :, :, 0:1] select?
A. The depth layer of the newest map for every example, keeping the layer axis. All batch items, last map in time, all rows and columns, first layer only.

Q. What does [None] do?
A. It adds a batch axis of length 1, so one map (4, H, W, 3) becomes (1, 4, H, W, 3).

Q. What does model(x, training=False).numpy()[0, :, :, 0] do?
A. It runs the model in prediction mode and takes the one example and its one layer, leaving a 2D depth map.

Q. What does set_random_seed(42) do?
A. It fixes the random numbers so a rerun gives the same result.

Q. What do np.load and np.save do?
A. They read an array from a file and save one to a file, and `mmap_mode="r"` reads a big file piece by piece.

Q. What do glob.glob and [::6] do?
A. `glob.glob` lists files matching a pattern, and `[::6]` keeps every sixth item, one map every 30 minutes from every 5.

Q. What does uniform_filter(size=15) do?
A. It replaces each square by the average of the 15 by 15 squares around it, used to get the local ground height.

Q. What does np.stack do?
A. It puts arrays on top of each other along a new axis, here depth, rain and height as 3 layers.

Q. What do compile, fit and predict do?
A. Compile chooses optimiser and loss, fit trains on the training tiles, predict runs the trained network on new inputs.

Q. What does set_weights(get_weights()) do?
A. It copies the learned numbers into a model with the same layers, for example one the size of a whole map.

Q. What does tf.GradientTape do?
A. It records the calculation so we can find gradients, which Grad-CAM needs.

Q. What does np.digitize do?
A. It turns a flooded share into a severity class from 0 to 3 using the limits 5, 15, 30%.

Q. What does confusion_matrix do?
A. It counts how many tiles of each true class got each answer, and the diagonal is correct.

Q. What are the harmless warnings?
A. `NotGeoreferencedWarning` means the map file has no position inside it, the HTTP 429 lines mean the server asked us to slow down, and the Adam warning means the saved optimiser memory is skipped.

3 Why this design? One-line answers for every choice in the model

Mam will ask about the code and the theory behind it, for example "why 16 filters?". Each answer below is one sentence you can say aloud, with a short note in small print for when she asks more. Tags: **Notebook** means the notebook or report says it, **Evidence** means one of our own experiments shows it, **General** means it is the usual reason and we did not test it. We never ran an experiment that changes the number of filters or layers of Model A, so never say "we tried 8 and 32".

Table 1. Model A layer by layer for a 64 by 64 tile (Model B has the same layers). Parameters match the notebook's `summary()`, total 62,833.

Name	Keras call	Output shape	Params	Job
`past_maps`	Input	4, 64, 64, 3	0	Four maps: depth, rain, local height.
`cnn_1`	TimeDistributed (Conv2D(16, 3, same, relu))	4, 64, 64, 16	448	16 simple patterns per map.
`cnn_2`	TimeDistributed (Conv2D(16, 3, same, relu))	4, 64, 64, 16	2,320	Bigger shapes from the simple ones.
`memory`	ConvLSTM2D(32, 3, same)	64, 64, 32	55,424	Reads the 4 maps in order, keeps 32 memory maps.
`decoder`	Conv2D(16, 3, same, relu)	64, 64, 16	4,624	Memory to 16 working features.
`depth_change`	Conv2D(1, 1), starts at zero	64, 64, 1	17	16 features to one number: the change in depth.
`add, next_depth`	Add, then ReLU	64, 64, 1	0	Last depth plus change, never below 0.

Counting parameters: kernel height $\times$ width $\times$ input layers $\times$ filters, plus one bias per filter. `cnn_1`: $3\cdot3\cdot3\cdot16+16=448$. `cnn_2`: $3\cdot3\cdot16\cdot16+16=2{,}320$. `decoder`: $3\cdot3\cdot32\cdot16+16=4{,}624$. `depth_change`: $16+1=17$. ConvLSTM: four parts (three gates and the candidate) over 16 input plus 32 memory layers, $4(3\cdot3\cdot48\cdot32+32)=55{,}424$, which is 88% of the model.

3.1 The layers

Q. Why a CNN and not a dense network?
A. A flood is a picture, and a convolution looks at neighbouring squares with the same small detector everywhere. Evidence: the original laptop model squeezed each frame through a dense layer of 128 numbers and scored only 0.06 to 0.19, while local convolutional networks scored 0.81 to 0.87.

Q. Why 16 filters?
A. It is a small standard size and each filter learns one pattern, but the notebook does not say why 16 was chosen and we did not tune it. General. Evidence: laptop networks of 4,101, 4,741 and 652,918 parameters all scored within about 0.03, so size was not the limit.

Q. Why a second convolution layer?
A. A second 3 by 3 layer sees a 5 by 5 area, so it finds bigger shapes such as river bends with fewer weights than one 5 by 5 filter. General, we did not test removing it. Two 3 by 3 layers use 18 weights per channel pair against 25, and add one more ReLU bend.

Q. Why a 3 by 3 kernel?
A. It is the smallest filter that sees a square and all eight neighbours, so it can see the direction of a slope. Evidence: kernels of 1, 5 and 11 scored within 0.025 in the laptop study, so neighbours add little.

Q. What does padding same do?
A. It adds a border of zeros so the output stays 64 by 64, which is needed to add the change to the last depth map square by square.

Q. Why ReLU, and why none on the last convolution?
A. ReLU zeroes negatives and gives the network the bend it needs, and the change in depth can be negative, so the last convolution has none. The final ReLU after the addition only keeps the depth from going below zero.

Q. What does TimeDistributed do?
A. It runs the same convolution, with the same weights, on each of the four maps separately. Shape (4, 64, 64, 3) becomes (4, 64, 64, 16), and sharing weights is why `cnn_1` costs 448 and not 4×448.

Q. Why a ConvLSTM with 32 filters?
A. We need to remember how the water moved over 1.5 hours, and a ConvLSTM keeps its memory as a map, using 32 memory layers. General for 32, not tuned. Evidence: an 8-filter ConvLSTM scored 0.866, 0.840, 0.811 in the laptop study, and one 140 times larger did not do better (these are not Models A and B).

Q. What does the ConvLSTM output?
A. By default only the memory after the last of the four steps, shape (64, 64, 32), which is all we need for one forecast.

Q. Why a decoder layer and a final 1 by 1 convolution?
A. The decoder turns the 32 memory maps into 16 features, and the 1 by 1 layer mixes them into one number per square, the predicted change, with 17 parameters.

Q. Why no pooling, dropout or batch normalisation?
A. Pooling would shrink the map and we need an answer for every square, the model is small, and early stopping handles overfitting. We did not try dropout or batch normalisation.

Q. Why does the last layer start at zero?
A. With zero weights the predicted change is zero, so before training the model exactly copies the last map. Notebook. In Model B the same trick starts it exactly at the trend forecast, and the notebook checks the difference is 0.0.

Q. Why predict the change and not the whole map?
A. In 30 minutes most squares hardly change, so learning the small difference is much easier than redrawing the map.

3.2 Data and training

Q. Why 4 input maps?
A. Four maps 30 minutes apart show the last 1.5 hours, enough to see whether the water is rising, falling or steady. It is the setting `INPUT_STEPS = 4`. We did not test other lengths in Part 1.

Q. Why 64 by 64 tiles, and why 4,000?
A. The Australia map is 1073 by 1073, too big to train in one piece, and a network of small filters can run on a whole map afterwards. 4,000 training, 600 checking, 1,000 test tiles. `forecaster_for_whole_map` copies the weights into a whole-map model.

Q. Why divide rain by 10 and use local ground height?
A. Rain is scaled to a similar size as the other inputs, and water flows because of height differences to the neighbours, so we use height minus the 15 by 15 average. Notebook.

Q. Why mean squared error?
A. Our skill score is built on squared error, so we train on what we are scored on, and big mistakes count more. Model B uses centimetres because, says the notebook, in metres the numbers are tiny.

Q. Why Adam, learning rate 0.001, batch 16, 20 epochs?
A. They are the usual safe settings, Adam adapts its own step size, and 20 is only a ceiling because early stopping ends training sooner. General. Evidence: in the laptop study a bigger batch with a higher rate cost 0.008 to 0.026 and longer training gained up to 0.024, both near seed noise of about 0.01.

Q. How does early stopping work?
A. `EarlyStopping(patience=4, restore_best_weights=True)` stops after 4 epochs without improvement on the 600 checking tiles and restores the best weights. For the 30 m Model A the checking error was lowest after the second epoch, then rose: overfitting.

Q. Why a 70, 15, 15 split in time?
A. Checking and test parts come after the training part, like a real forecast, and beta uses only the training part, so there is no leakage.

Q. Why two models, 30 m and 480 m?
A. A 30 m and a 480 m square behave very differently, so we never mix them: Australia to UK, and Pakistan to Mozambique.

Q. Why not a Transformer or U-Net?
A. Our project was a CNN with an LSTM memory, and the dataset's own benchmark has U-Net and FNO but no recurrent model. Evidence: a local CNN with no recurrence scored the same at equal training length ($+0.004$ and -0.014, inside the ± 0.03 band).

3.3 Trend forecast, Model B and the water rule

Q. How is beta found?
A. It is the least-squares number, the sum of previous change times next change divided by the sum of previous change squared, giving 0.921 for 30 m and 0.948 for 480 m. That is `top / bottom` in `fit_trend_factor`. Below 1 means the change carries on but slows.

Q. What do Subtract and Rescaling do in Model B?
A. Subtract gives the last change, Rescaling(beta) scales it, Add and ReLU make the trend forecast, and the network's output times 0.01 is added as a correction in centimetres.

Q. Why does Model B not beat the trend?
A. At 30 minutes the trend already removes 91 to 99% of copying's squared error, leaving only 1, 5, 9 and 6%. If training does not lower the checking error the code keeps the plain trend (`if best_loss >= start_loss`).

Q. How does the water-balance rule work?
A. Average rise should equal rain times 0.5 hours, so the rule adds missing water to every square or removes extra water only from wet squares. It fixes the total only, the network still decides where the water goes.

3.4 Capsule network, Grad-CAM and the labels

Q. Why a capsule network?
A. It was in our proposal, and a capsule's length says how sure the network is, which suits a severity rating. It did not beat a plain layer once we used augmentation, and the forecast map's own class is right 99% of the time.

Q. Why is the ConvLSTM frozen in the capsule model?
A. We reuse the forecaster's memory and train only the new layers, which is faster and needs fewer examples. General. It also lets one Grad-CAM function explain both models.

Q. Why strided convolutions and 256 primary capsules?
A. Each stride-2 convolution halves the map (64, 32, 16, 8) and $8 \times 8 \times 4$ types gives 256 capsules of 8 numbers. The notebook gives no reason for the kernel sizes (5, 5, 3) or the count, general reasoning only.

Q. What does squash do?
A. It shrinks a vector's length to between 0 and 1 and keeps its direction, so length can mean confidence. Formula in the code: $\frac{|v|^2}{1+|v|^2} \cdot \frac{v}{|v|}$.

Q. What is routing by agreement?
A. Each small capsule votes for each class, votes that agree with the result get more weight next round, and the code repeats this 3 times. The vote weights are $256 \times 4 \times 16 \times 8 = 131{,}072$ numbers, with a softmax over the 4 classes.

Q. What is the margin loss?
A. The true class capsule should be longer than 0.9 and the others shorter than 0.1, with the others weighted 0.5. Same constants as Sabour et al., not tuned by us. The notebook comment says class weights stop it answering None every time.

Q. Why Grad-CAM at the ConvLSTM memory?
A. It is the last layer that is still a map and is shared by the forecaster and the capsule network. General. Steps: average gradient per memory map, weighted sum, keep positive part, scale 0 to 1.

Q. Why 30 cm and the 5, 15, 30% limits?
A. They are our own definition: severity is the share of normally dry land under more than 30 cm, below 5% None, 5 to 15 Minor, 15 to 30 Moderate, 30 and up Severe. Notebook. Not an official standard. The old deepest-square label did not measure the flood.

3.5 Formulas worth knowing

Convolution: $y = \max\left(0,\ b + \sum w x\right)$ summed over the 3 by 3 squares and all input layers, with the same weights at every position. Output size per side $\lfloor (n + 2p - k)/s \rfloor + 1$ (map n, kernel k, padding p, stride s), so "same" padding keeps n and stride 2 halves it.

LSTM cell: $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$ forget gate, $i_t = \sigma(W_i[\cdot] + b_i)$ input gate, $o_t = \sigma(W_o[\cdot] + b_o)$ output gate, $g_t = \tanh(W_g[\cdot] + b_g)$ candidate, then $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and $h_t = o_t \odot \tanh(c_t)$. The memory is updated by adding, so the learning signal does not fade over many steps (the vanishing gradient problem of a plain recurrent network).

ConvLSTM: the same equations, but x, h, c are maps and every product with W is a convolution, for example $f_t = \sigma(W_{xf} * x_t + W_{hf} * h_{t-1} + b_f)$. Keras defaults (tanh and sigmoid) are unchanged.

Others: MSE $= \frac{1}{N}\sum(\hat{y} - y)^2$. Skill $= 1 - \sum(\hat{y} - y)^2 / \sum(\text{last} - y)^2$. Trend $= \max(0,\ d_t + \beta(d_t - d_{t-1}))$. Softmax $e^{s_k} / \sum_j e^{s_j}$. Adam moves each weight by about learning rate $\times$ average gradient $/ \sqrt{\text{average squared gradient}}$. Grad-CAM $= \text{ReLU}(\sum_k \alpha_k A_k)$ with A_k memory map k and α_k its average gradient.

What each square sees: two 3 by 3 convolutions give 5 by 5, the ConvLSTM and the decoder each add one more step, so each forecast square depends on at least a 9 by 9 patch of the newest map (at least 270 m at 30 m and 4.3 km at 480 m).

3.6 Five answers to learn by heart

1. **Why this architecture?** The CNN reads the water's shape in each map, the ConvLSTM remembers how it moved over 1.5 hours, and a decoder turns the memory into the change in depth.
2. **Why 16 filters and 32 memory maps?** Small standard sizes that we did not tune, and our laptop study suggests size is not what limits the skill.
3. **Why a second convolution?** It sees a 5 by 5 area, so it finds bigger shapes with fewer weights than one big filter, and we did not test removing it.
4. **Why predict the change, last layer at zero?** Most squares hardly change in 30 minutes, so the model starts as a perfect copy and learns only the small difference.
5. **Main finding about the design?** A trend forecast with one number matches the network at 30 minutes, so what matters is each square's recent depth history, not the layer design.